9h. Mutations and Pre-Allocations

Martin Alfaro
PhD in Economics

INTRODUCTION

Certain operations relying on for-loops could involve the creation of new vectors during each iteration, leading to repeated memory allocation. In many cases, this dynamic allocation may be unnecessary, particularly if these vectors hold temporary intermediate results that don't need to be preserved beyond the current iteration.

A similar issue arises when the same set of operations must be executed repeatedly, which typically occurs in simulations and Monte Carlo experiements. In these situations, the computations are identical, but they must be performed multiple times with different inputs. In case that only the final output needs to be retained, generating new intermediate vectors for every run results in avoidable allocations.

In both scenarios, performance can be improved through **pre-allocation**. This technique involves initializing a vector before execution begins, and then reuse it throughout the computation by mutating its elements. By allocating memory upfront and updating the vector in place, the program avoids the overhead of memory allocation.

Remarkably, the technique isn't exclusive to Julia, but rather represents an optimization strategy applicable across programming languages. Its effectiveness ultimately stems from **favoring mutations over the creation of new objects**, thus minimizing allocations on the heap.

Our presentation will include a review of methods for initializing vectors, which is a prerequisite for implementing a pre-allocation strategy. We then present some scenarios where pre-allocation proves advantageous, with special emphasis on its advantages within nested for-loops.

THE ADVANTAGES OF MUTATING VECTORS

The benefits of pre-allocation arise from the broader advantages of using in-place operations. As noted earlier, this approach encourages reusing existing vectors whenever possible, rather than creating new objects. Because this principle is valuable beyond the context of pre-allocation, we begin by illustrating the performance gains that come from reusing the same memory address.

For instance, this principle guides how we should handle operations such as sorting a vector `x`: a mutating approach should be preferred preferable. The following example demonstrates the performance benefits of doing so.

SORT

```
x = rand(1_000_000)
```

```
julia> @btime sort($x)
  19.711 ms (9 allocations: 15.267 MiB)
```

SORT!

```
x = rand(1_000_000)
```

```
julia> @btime sort!($x)
  1.067 ms (0 allocations: 0 bytes)
```

As we can see, the differences in runtime are substantial. If the same operation were executed repeatedly inside a for-loop, these performance gaps would become even more pronounced.

Pre-allocation builds on the same idea, but applies to scenarios where a vector like `x` doesn't yet exist. Rather, the scenario requires initializing a vector and then reuse it throughout a for-loop or repeated executions of a program. With this in mind, we begin with a review of how to initialize vectors.

INITIALIZING VECTORS

> **Remark**
> The review of methods for vector initialization will be relatively brief and centered on performance considerations. For a more detailed coverage, see the **section about vector initialization**, as well as the sections on **in-place assignments** and **in-place functions**.

Vector initialization refers to the process of creating a vector that subsequently will be filled with values. The process typically involves two steps: reserving space in memory and populating that space with some initial values. An efficient approach to initializing a vector involves performing only the first step, keeping whatever content is in the memory address. These values held in memory are referred to as `undef` (undefined). Although Julia will display them as numerical values, they're essentially arbitrary and meaningless.

There are two methods for initializing a vector with `undef` values. The first one is through a **constructor**, requiring the specification of the length and element types. The second one is based on the function `similar`, which creates a vector with the same type and dimension as another existing vector. This approach is particularly useful when the output sought matches the structure of an input.

Below, we compare the performance of several approaches to initializing vectors. In particular, we establish that working with `undef` values is faster than populating vectors with specific values. To starkly show the differences in execution time, we repeat the process of vector creation 100,000 times.

UNDEF

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        Vector{Int64}(undef, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  2.224 ms (100000 allocations: 85.449 MiB)
```

"SIMILAR" (UNDEF)

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        similar(x)
    end
end
```

```
julia> @btime foo($x, $repetitions)
  2.421 ms (100000 allocations: 85.449 MiB)
```

ZEROS

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        zeros(Int64, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  9.393 ms (100000 allocations: 85.449 MiB)
```

ONES

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        ones(Int64, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  9.504 ms (100000 allocations: 85.449 MiB)
```

FILL

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        fill(2, length(x))                  # vector filled with integer 2
    end
end
```

```
julia> @btime foo($x, $repetitions)
  9.762 ms (100000 allocations: 85.449 MiB)
```

> **Remark**
>
> Recall that `_` isn't a keyword, but rather a convention widely adopted by programmers to denote **dummy variables**. These are variables included solely to meet syntactical requirements, but are never actually used or referenced within the code. In our example, the inclusion of `_` is because for-loops must always include a variable to iterate over. While any other symbol could be used, `_` signals programmers that its value can be safely ignored.

INITIALIZING VECTORS IN FUNCTIONS

We can initialize a vector by passing it to the function as a keyword argument. This strategy even enables the use of `similar(x)`, where `x` is a previous function argument. Considering this, the following two implementations are equivalent.

IMPLEMENTATION 1

```
function foo(x)
   output = similar(x)
   # <some calculations using 'output'>
end
```

IMPLEMENTATION 2

```
function foo(x; output = similar(x))

   # <some calculations using 'output'>
end
```

When multiple variables of the same type need to be initialized, array comprehensions offer a concise solution. A more efficient alternative is based on the so-called **generators**, which are the lazy counterpart of array comprehensions. This reduces memory allocations by initializing and assigning the vectors element-wise. The behavior differs from array comprehensions, which first create a vector that holds all the initialized vectors.

Below we illustrate these strategies. Although the example is based on `similar(x)`, note that the same principle applies a constructor like `Vector{Float64}(undef, length(x))`.

ARRAY COMPREHENSION

```
x = [1,2,3]

function foo(x)
    a,b,c = [similar(x) for _ in 1:3]
    # <some calculations using a,b,c>
end
```

```
julia> @btime foo($x)
  62.042 ns (8 allocations: 320 bytes)
```

GENERATOR

```
x = [1,2,3]

function foo(x)
    a,b,c = (similar(x) for _ in 1:3)
    # <some calculations using a,b,c>
end
```

```
julia> @btime foo($x)
  25.350 ns (3 allocations: 144 bytes)
```

DESCRIBING THE PRE-ALLOCATION TECHNIQUE

To describe how pre-allocation works, we'll consider a typical scenario where the strategy proves to be advantageous. The setting involves a **nested for-loop**, in which the output of a for-loop serves as an intermediate input for another for-loop. The example will rely on explicit for-loops, but also on expressions that internally compute the operation via a for-loop (e.g., broadcasting).

Let's first describe the inner for-loop that will be eventually embedded in an outer for-loop. Suppose we're assessing a worker's performance over a 30-day period, with daily scores recorded on a scale from 0 to 1. Defining successful performance as any score above 0.5, the following code snippets generate vectors indicating the days on which the goal was achieved.

BROADCASTING

```
nr_days            = 30
score              = rand(nr_days)

performance(score) = score .> 0.5
```

```
julia> @btime performance($score)
  46.611 ns (3 allocations: 96 bytes)
```

FOR-LOOP (EQUIVALENT)

```
nr_days            = 30
score              = rand(nr_days)

function performance(score)
    target = similar(score)

    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end
```

```
julia> @btime performance($score)
  31.296 ns (2 allocations: 304 bytes)
```

FOR-LOOP (EQUIVALENT)

```
nr_days            = 30
score              = rand(nr_days)

function performance(score; target=similar(score))


    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end
```

```
julia> @btime performance($score)
  33.185 ns (2 allocations: 304 bytes)
```

Consider now that the output of this operation (namely, the vector of days in which the target was met) represents an intermediate step in another for-loop. For instance, suppose we have multiple workers, each with recorded performance scores. The goal is to summarize each worker's information through a summary statistic. In particular, the ratio of the standard deviation to the mean, which expresses variability in units of the average.

In practice, this statistic requires computing both the mean and the standard deviation. Below, we show that computing each statistic via reductions isn't efficient. The reason is that, while reductions avoid memory allocations, they require computing the days on which the target was met twice. Instead, it's more efficient to compute and store this vector beforehand, which we then reuse as an input for computing the mean and the standard deviation.

The result highlights a more general principle: when the same intermediate input is required for multiple outputs, the cost of allocating memory for the intermediate vector tends to be lower than its repeated computation. In the example, the reduction is implemented via **lazy broadcasting**, which internally relies on reduction techniques. The rest implement the result via explicit or implicit for-loops, with the intermediate input stored in a vector.

REDUCTIONS (INEFFICIENT)

```julia
nr_days             = 30
scores              = [rand(nr_days), rand(nr_days), rand(nr_days)]  # 3 workers



function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)

      stats[col] = std(@~ scores[col] .> 0.5) / mean(@~ scores[col] .> 0.5)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  456.721 ns (2 allocations: 80 bytes)
```

IMPLICIT NESTED FOR-LOOP

```julia
nr_days            = 30
scores             = [rand(nr_days), rand(nr_days), rand(nr_days)]  # 3 workers

performance(score) = score .> 0.5

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  391.280 ns (11 allocations: 368 bytes)
```

EXPLICIT NESTED FOR-LOOP

```julia
nr_days            = 30
scores             = [rand(nr_days), rand(nr_days), rand(nr_days)]   # 3 workers

function performance(score)
    target = similar(score)

    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  181.921 ns (8 allocations: 992 bytes)
```

EXPLICIT NESTED FOR-LOOP

```julia
nr_days             = 30
scores              = [rand(nr_days), rand(nr_days), rand(nr_days)]   # 3 workers

function performance(score; target = similar(score))


    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  219.670 ns (8 allocations: 992 bytes)
```

Once we establish that storing the intermediate vector is more performant, our approach will necessarily involve memory allocations. The methods described above, however, allocate more than required: during each iteration, a new vector is created for each worker to store the days in which the target was met. This intermediate vector doesn't need to be preserved for future use. Indeed, because it's stored in a local variable, it'll be discarded once the iteration concludes.

Such cases are strong candidates for pre-allocating intermediate results. By defining a single vector `target`, we can reuse it across iterations to compute the days when the target was met for each worker. This strategy incurs the overhead of creating the vector only once, after which it's reused for every worker.

The implementation requires defining the vector `target` before the execution of the outer for-loop. During each iteration, we then mutate target using an in-place function, which we call `performance!`. This function can update the contents either with a standard for-loop or by applying the broadcasting operator `.=`.

NO PRE-ALLOCATION

```julia
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]


performance(score) = score .> 0.5

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        target     = performance(scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```julia
julia> @btime repeated_call($scores)
  322.167 ns (11 allocations: 368 bytes)
```

PRE-ALLOCATION (VIA BROADCASTING)

```julia
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

performance!(target, score) = (@. target = score > 0.5)

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        performance!(target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```julia
julia> @btime repeated_call!($target, $scores)
  243.123 ns (2 allocations: 80 bytes)
```

PRE-ALLOCATION (VIA FOR-LOOP)

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function performance!(target, score)
    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end
end

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        performance!(target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  227.700 ns (2 allocations: 80 bytes)
```

⚠️ **Warning!** - Use of @. to update values

When your goal is to update the values of a vector, recall that `@.` has to be *placed at the beginning* of the statement.

NEW VARIABLE (NOT AN UPDATE)

```
# the following are equivalent and define a new variable
   output  = @. 2  * x
   output  =    2 .* x
```

AN UPDATE

```
# the following are equivalent and update 'output'
@. output  = 2  * x
   output .= 2 .* x
```

Compared to a for-loop, the method using `.=` provides a simpler syntax. This is why, when the function `performance!` is simple enough as in our example, it's common to avoid its definition and directly insert it into the inner for-loop. The possibility also enables implementing the update via a built-in in-place function. For instance, the function `map!` can be used with this purpose.

NO PRE-ALLOCATION

```julia
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]


function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        target     = @. score > 0.5
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  630.405 ns (23 allocations: 608 bytes)
```

PRE-ALLOCATION (VIA BROADCASTING)

```julia
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        @. target  = scores[col] > 0.5
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  241.027 ns (2 allocations: 80 bytes)
```

PRE-ALLOCATION (VIA MAP!)

```julia
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        map!(a -> a > 0.5, target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  220.948 ns (2 allocations: 80 bytes)
```